

Autoscaling

Autoscaling is the process of dynamically allocating resources to match performance requirements. As the volume of work grows, an application might need more resources to maintain the desired performance levels and satisfy service-level agreements (SLAs). As demand slackens and the extra resources are no longer needed, they can be de-allocated to minimize costs.

Autoscaling takes advantage of the elasticity of cloud-hosted environments while easing management overhead. It reduces the need for an operator to continually monitor the performance of a system and make decisions about adding or removing resources.

There are two main ways that an application can scale:

- **Vertical scaling**, also called scaling up and down, means to change the capacity of a resource. For example, you could move an application to a larger virtual machine size. Vertical scaling often requires making the system temporarily unavailable while it's being redeployed. Therefore, it's less common to automate vertical scaling.
- **Horizontal scaling**, also called scaling out and in, means adding or removing instances of a resource. The application continues running without interruption as new resources are provisioned. When the provisioning process is complete, the solution is deployed on these extra resources. If demand drops, the extra resources can be shut down cleanly and deallocated.

Many cloud-based systems, including Microsoft Azure, support automatic horizontal scaling. The rest of this article focuses on horizontal scaling.

ⓘ Note

Autoscaling mostly applies to compute resources. While it's possible to horizontally scale a database or message queue, this process usually involves [data partitioning](#), which is generally not automated.

Autoscaling components

An autoscaling strategy typically involves the following components:

- Instrumentation and monitoring systems at the application, service, and infrastructure levels. These systems capture key metrics, such as response times, queue lengths, CPU utilization, and memory usage.

- Decision-making logic evaluates these live usage metrics against predefined thresholds or schedules and decides whether to scale.
- Components and mechanisms perform the scaling action. Ideally, these components and mechanisms should be decoupled from the workload code itself and managed as an external process. Code that's idle or being overwhelmed shouldn't be responsible for scaling itself.
- Testing, monitoring, and tuning capabilities for the autoscaling strategy to ensure that it functions as expected.

Azure provides built-in autoscaling mechanisms that address common scenarios. If a particular service or technology doesn't have built-in autoscaling functionality or if you have specific autoscaling requirements beyond its capabilities, consider a custom implementation. A custom implementation collects operational and system metrics, analyzes the metrics, and scales resources accordingly.

Configure autoscaling for an Azure solution

Azure provides built-in autoscaling for most compute options.

- **Azure virtual machines** autoscale via [virtual machine scale sets](#), which manage a set of virtual machines as a group. For more information, see [Use automatic scaling and virtual machine scale sets](#).
- **Azure Service Fabric** supports autoscaling through virtual machine scale sets. Every node type in a Service Fabric cluster is set up as a separate virtual machine scale set. Each node type can be scaled in or out independently. For more information, see [Scale a Service Fabric cluster in or out by using autoscale rules](#).
- **Azure App Service** has built-in autoscaling. Autoscale settings apply to all of the apps within an app service. For more information, see [Scale instance count manually or automatically](#) and [Scale up an app in App Service](#).

These compute options all use the [Azure Monitor autoscale feature](#) to provide a common set of autoscaling functionality.

- **Azure Functions** differs from the previous compute options because you don't need to configure any autoscale rules. Instead, Azure Functions automatically allocates compute power when your code runs. Azure Functions scales out as necessary to handle load. For more information, see [Choose the correct hosting plan for Azure Functions](#).

A custom autoscaling solution can sometimes be useful. For example, you could use Azure Diagnostics and application-based metrics, along with custom code to monitor and export the

application metrics. Then you could define custom rules based on these metrics, and use Azure Resource Manager REST APIs to trigger autoscaling. However, a custom solution isn't simple to implement and should be considered only if none of the previous approaches can fulfill your requirements.

Use the built-in autoscaling features of the platform if they meet your requirements. If not, carefully consider whether you need more complex scaling features. Examples of other requirements might include more granularity of control, different ways to detect trigger events for scaling, scaling across subscriptions, and scaling other types of resources.

Use the Azure Monitor autoscale feature

The [Azure Monitor autoscale feature](#) provides a common set of autoscaling functionality for virtual machine scale sets, App Service, and Azure Cloud Services. Scaling can be performed on a schedule or based on a runtime metric, such as CPU or memory usage.

Consider the following examples:

- Scale out to 10 instances on weekdays, and scale in to four instances on Saturday and Sunday.
- Scale out by one instance if average CPU usage is above 70%, and scale in by one instance if CPU usage falls below 50%.
- Scale out by one instance if the number of messages in a queue exceeds a certain threshold.

Scale up the resource when load increases to ensure availability. At times of low usage, scale down so you can optimize cost. Always use a scale-out and scale-in rule combination. Otherwise, the autoscaling takes place only in one direction until it reaches the threshold (maximum or minimum instance counts) set in the profile.

Select a default instance count that's safe for your workload. The resource scales based on that value if maximum or minimum instance counts aren't set.

For a list of built-in metrics, see [Azure Monitor autoscaling common metrics](#). You can also implement custom metrics by using Application Insights.

You can configure autoscaling by using PowerShell, the Azure CLI, an Azure Resource Manager template, or the Azure portal. For more detailed control, use the [Resource Manager REST API](#). The [Azure Monitoring Management Library](#)[↗] and the [Microsoft Insights Library](#)[↗] (in preview) are SDKs that allow collecting metrics from different resources and perform autoscaling by making use of the REST APIs. For resources where Resource Manager support isn't available, or

if you use Azure Cloud Services, the Service Management REST API can be used for autoscaling. In all other cases, use Resource Manager.

Consider the following points when using autoscaling:

- Consider whether you can predict the load on the application accurately enough to use scheduled autoscaling, adding and removing instances to meet anticipated peaks in demand. If you can't, use reactive autoscaling based on runtime metrics to handle unpredictable changes in demand. Typically, you can combine these approaches.

For example, create a strategy that adds resources based on a schedule of the times when you know the application is busiest. Extra resources help ensure that capacity is available when required, without any delay from starting new instances. For each scheduled rule, define metrics that allow reactive autoscaling during that period to ensure that the application can handle sustained but unpredictable peaks in demand.

- It's often difficult to understand the relationship between metrics and capacity requirements, especially when an application is initially deployed. Configure a little extra capacity at the beginning, and then monitor and tune the autoscaling rules to bring the capacity closer to the actual load.
- Configure the autoscaling rules, and then monitor the performance of your application over time. Use the results of this monitoring to adjust the way in which the system scales if necessary. However, keep in mind that autoscaling isn't an instantaneous process. It takes time to react to a metric such as average CPU utilization exceeding or falling below a specified threshold.
- Autoscaling rules that use a detection mechanism based on a measured trigger attribute use an aggregated value over time, rather than instantaneous values, to trigger an autoscaling action. Trigger attributes include CPU usage or queue length. By default, the aggregate is an average of the values. This approach prevents the system from reacting too quickly or causing rapid oscillation. It also allows time for new instances that are automatically started to settle into running mode. Other autoscaling actions can't occur while the new instances are starting up. For Azure Cloud Services and Azure Virtual Machines, the default period for the aggregation is 45 minutes. So it can take up to this period of time for the metric to trigger autoscaling in response to spikes in demand. You can change the aggregation period by using the SDK, but periods of less than 25 minutes might cause unpredictable results. For the Web Apps feature of App Service, the averaging period is shorter, allowing new instances to be available in about five minutes after a change to the average trigger measure.
- Avoid *flapping* where scale-in and scale-out actions continually go back and forth. Suppose there are two instances. The upper limit is 80% CPU and the lower limit is 60%.

When the load is at 85%, another instance is added. After some time, the load decreases to 60%. Before the autoscale service scales in, it calculates the distribution of total load (of three instances) when an instance is removed, taking it to 90%. It would have to scale out again immediately. So, it skips scaling in and you might never see the expected scaling results.

The flapping situation can be controlled by choosing an adequate margin between the scale-out and scale-in thresholds.

- Manual scaling resets based on the maximum and minimum number of instances used for autoscaling. If you manually update the instance count to a value higher or lower than the maximum value, the autoscale engine automatically scales back to the minimum (if lower) or the maximum (if higher). For example, you set the range between three and six. If you have one running instance, the autoscale engine scales to three instances on its next run. Likewise, if you manually set the scale to eight instances, on the next run autoscale scales it back to six instances on its next run. Manual scaling is temporary unless you reset the autoscale rules as well.
- The autoscale engine processes only one profile at a time. If a condition isn't met, it checks for the next profile. Keep key metrics out of the default profile because that profile is checked last. Within a profile, you can have multiple rules. On scale-out, autoscale runs if any rule is met. On scale-in, autoscale requires all rules to be met.

For more information about how Azure Monitor scales, see [Best practices for autoscale](#).

- If you configure autoscaling by using the SDK rather than the portal, you can specify a more detailed schedule during which the rules are active. You can also create your own metrics and use them with or without any of the existing ones in your autoscaling rules. For example, you might wish to use alternative counters, such as the number of requests per second or the average memory availability. Or you might use custom counters to measure specific business processes.
- When you autoscale Service Fabric, the node types in your cluster are made of virtual machine scale sets at the back end, so you need to set up autoscale rules for each node type. Take into account the number of nodes that you must have before you set up autoscaling. Your reliability level drives the minimum number of nodes that you must have for the primary node type. For more information, see [Scale a Service Fabric cluster in or out by using autoscale rules](#).
- You can use the portal to link resources such as Azure SQL Database instances and queues to a cloud service instance. This method allows you to more easily access the separate manual and automatic scaling configuration options for each of the linked resources. For more information, see [Manage Azure Cloud Services](#).

- When you configure multiple policies and rules, they could conflict with each other. Autoscale uses the following conflict resolution rules to ensure that there's always a sufficient number of instances running:
 - Scale-out operations always take precedence over scale-in operations.
 - When scale-out operations conflict, the rule that initiates the largest increase in the number of instances takes precedence.
 - When scale-in operations conflict, the rule that initiates the smallest decrease in the number of instances takes precedence.
- In an App Service Environment, any worker pool or front-end metrics can be used to define autoscale rules. For more information, see [App Service Environment overview](#).

Application design considerations

Autoscaling isn't an instant solution. Simply adding resources to a system or running more instances of a process doesn't guarantee that the performance of the system improves.

Consider the following points when designing an autoscaling strategy:

- The system must be designed to be horizontally scalable. Avoid making assumptions about instance affinity. Don't design solutions that require that the code is always running in a specific instance of a process. When scaling a cloud service or website horizontally, don't assume that a series of requests from the same source are always routed to the same instance. For the same reason, design services to be stateless to avoid requiring a series of requests from an application to always be routed to the same instance of a service. When designing a service that reads messages from a queue and processes them, don't make any assumptions about which instance of the service handles a specific message. Autoscaling could start more instances of a service as the queue length grows. The [Competing Consumers pattern](#) describes how to handle this scenario.
- If the solution implements a long-running task, design this task to support both scaling out and scaling in. Without proper design, such a task could prevent an instance of a process from being shut down cleanly when the system scales in. Or it could lose data if the process is forcibly terminated. Ideally, refactor a long-running task and break up the processing that it performs into smaller, discrete chunks. For an example, see [Pipes and Filters pattern](#).
- Alternatively, you can implement a checkpoint mechanism that records state information about the task at regular intervals. Save this state information in durable storage that any instance of the process that runs the task can access. So if the process is shut down, the work that it was performing can be resumed from the last checkpoint by using another

instance. There are libraries that provide this functionality, such as [NServiceBus](#) and [MassTransit](#). They transparently persist state, where the intervals are aligned with the processing of messages from queues in Azure Service Bus.

- When background tasks run on separate compute instances, such as in worker roles of a cloud services-hosted application, you might need to scale different parts of the application by using different scaling policies. For example, you might need to deploy more user interface (UI) compute instances without increasing the number of background compute instances, or the opposite. You can offer different levels of service, such as basic and premium service packages. You might need to scale out the compute resources for premium service packages more aggressively than resources for basic service packages. This approach helps you meet SLAs.

Other scaling criteria

- Consider the length of the queue over which UI and background compute instances communicate. Use it as a criterion for your autoscaling strategy. This criteria can indicate an imbalance or difference between the current load and the processing capacity of the background task. There's a slightly more complex but better attribute to base scaling decisions on. Use the time between when a message was sent and when its processing was complete, known as the *critical time*. If this critical time value is below a meaningful business threshold, then it's unnecessary to scale, even if the queue length is long.
 - For example, there could be 50,000 messages in a queue. But the critical time of the oldest message is 500 ms, and that endpoint is dealing with integration with a partner web service for sending out emails. Business stakeholders might not consider this scenario as urgent enough to justify the cost of scaling out.
 - On the other hand, there could be 500 messages in a queue, with the same 500-ms critical time. But the endpoint is part of the critical path in a real-time online game, where business stakeholders defined a 100-ms or less response time. In that case, scaling out makes sense.
 - In order to make use of critical time in autoscaling decisions, it's helpful to have a library automatically add the relevant information to the headers of messages during transmission and processing. One such library that provides this functionality is [NServiceBus](#).
- If you base your autoscaling strategy on counters that measure business processes, ensure that you fully understand the relationship between the results from these types of counters and the actual compute capacity requirements. Examples of counters include the number of orders placed every hour or the average runtime of a complex transaction.

It might be necessary to scale more than one component or compute unit in response to changes in business process counters.

- To prevent a system from attempting to scale out excessively, consider limiting the maximum number of instances that can be automatically added. This approach also avoids the costs associated with running many thousands of instances. Most autoscaling mechanisms allow you to specify the minimum and maximum number of instances for a rule. In addition, consider gracefully degrading the functionality that the system provides if you deploy the maximum number of instances and the system is still overloaded.
- Keep in mind that autoscaling might not be the most appropriate mechanism to handle a sudden burst in workload. It takes time to set up and start new instances of a service or add resources to a system. And the peak demand might pass by the time these extra resources are available. In this scenario, it might be better to throttle the service. For more information, see [Throttling pattern](#).
- Conversely, if you need the capacity to process all requests when the volume fluctuates rapidly, consider using an aggressive autoscaling strategy that starts extra instances more quickly. Ensure that cost isn't a major contributing factor. You can also use a scheduled policy that starts a sufficient number of instances to meet the maximum load before that load is expected.
- The autoscaling mechanism should monitor the autoscaling process and log the details of each autoscaling event. These details include what triggered the event, what resources were added or removed, and when it occurred. If you create a custom autoscaling mechanism, ensure that it incorporates this capability. Analyze the information to help measure the effectiveness of the autoscaling strategy, and tune it if necessary. You can tune both in the short term, as the usage patterns become more obvious, and over the long term, as the business expands or the requirements of the application evolve. If an application reaches the upper limit defined for autoscaling, the mechanism might also alert an operator who could manually start extra resources if necessary. Under these circumstances, the operator might also be responsible for manually removing these resources after the workload eases.

Related resources

The following patterns and guidance might also be relevant to your scenario when implementing autoscaling:

- The [Throttling pattern](#) describes how an application can continue to function and meet SLAs when an increase in demand places an extreme load on resources. Throttling can be

used with autoscaling to prevent a system from being overwhelmed while the system scales out.

- The [Competing Consumers pattern](#) describes how to implement a pool of service instances that can handle messages from any application instance. Autoscaling can be used to start and stop service instances to match the anticipated workload. This approach enables a system to process multiple messages concurrently to optimize throughput, improve scalability and availability, and balance the workload.
- [Monitoring and diagnostics](#), including instrumentation and metrics, are vital for gathering the information that can drive the autoscaling process.

Last updated on 12/17/2022